



Qiskit Summer Dojo: Qiskit v2.X資格対策 #1

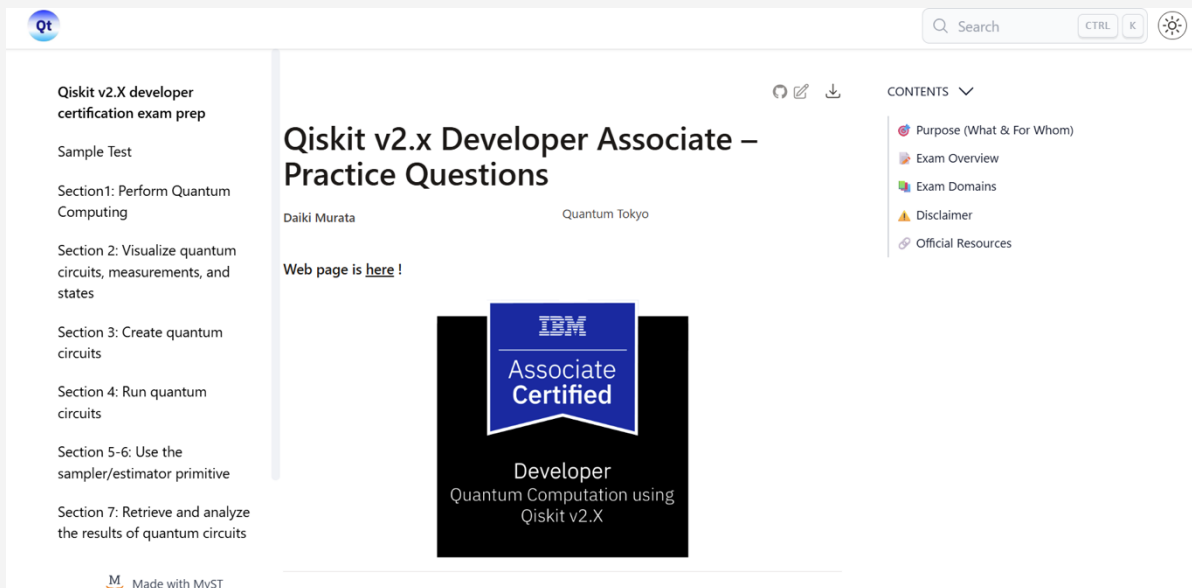
～量子操作 & 可視化編～

2026/08/07

Ayumu Shiraishi



- 「Qiskit v2.x Developer Associate – Practice Questions」に掲載されている内容について重要ポイントの紹介とそれに紐づく問題の解説を行います。
- 問題はこちら : <https://quantum-tokyo.github.io/qiskit-certification-prep/>
- 例題を出した時に少し時間を取りますので、正解がどれかを考えてみてください。
- 本セッションでは時間の関係で全ての問題の解説は行いませんので、ご了承ください。



- **Section1: Perform Quantum Computing**
 - TASK 1.1: Define Pauli Operators
 - ポイント
 - 例題
 - TASK 1.2: Apply quantum operations
 - ポイント
 - 例題
- **Section 2: Visualize quantum circuits, measurements, and states**
 - TASK 2.1: Visualize quantum circuits
 - ポイント
 - 例題
 - TASK 2.2: Visualize quantum measurements
 - ポイント
 - 例題
 - TASK 2.3: Visualize quantum states
 - ポイント
 - 例題

- **Section1: Perform Quantum Computing**
 - TASK 1.1: Define Pauli Operators

TASK 1.1: Define Pauli Operators ポイント

Pauli X : ビットフリップ (NOT演算)
Pauli Z : 位相フリップ
Pauli Y : ビットフリップ + 位相フリップ

$$X = \sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, Y = \sigma_y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, Z = \sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Pauliクラス : パウリ演算子を生成するクラス

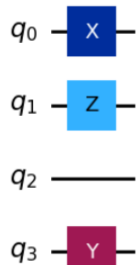
使い方例 :

- Pauli("YIZX") : 4ビットにそれぞれに対して、0番目ビットから順にX,Z,I,Yをかける
- Pauli([0,1,0,1], [0,0,1,1]) : 4ビットに対して0番目ビットから順にI,Z,X,Yをかける
1/0でもTrue/False可

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Pauli

qc = QuantumCircuit(4)
p = Pauli("YIZX")

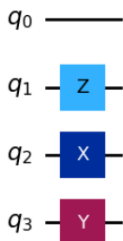
# 回路に追加 (第1引数に演算子、第2引数に適用する量子ビットのインデックス)
qc.append(p, [0, 1, 2, 3])
qc.decompose().draw('mpl')
```



```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Pauli

qc = QuantumCircuit(4)
z_bits=[0,1,0,1]
x_bits=[0,0,1,1]
p = Pauli((z_bits,x_bits))

# 回路に追加 (第1引数に演算子、第2引数に適用する量子ビットのインデックス)
qc.append(p, [0, 1, 2, 3])
qc.decompose().draw('mpl')
```



ビットの組み合わせによるPauli演算子の生成ルール

Zbit	Xbit	Pauli
0	0	I
1	0	Z
0	1	X
1	1	Y

2. Which of the following input formats are valid when creating a `Pauli` object?

- a. `Pauli("IXYZ")`
- b. `Pauli([1, 0, 1, 1])`
- c. `Pauli((z_bits, x_bits))`
- d. `Pauli(np.array([[0, 1],[1, 0]]))`

TASK 1.1: Define Pauli Operators 正解

2. Which of the following input formats are valid when creating a `Pauli` object?

☒ a. `Pauli("IXYZ")`

☐ b. `Pauli([1, 0, 1, 1])`

同じサイズの 1 次元配列（ベクトル）が 1 つではNG

☒ c. `Pauli((z_bits, x_bits))`

☐ d. `Pauli(np.array([[0, 1],[1, 0]]))`

2 次元配列は入力不可

- **Section1: Perform Quantum Computing**
 - TASK 1.2: Apply quantum operations

TASK 1.2: Apply quantum operations ポイント

既存回路にゲート（回路）を追加する方法が複数ある

追加方法（記述例）	特徴
<code>qc.x(1), qc.h([0,1]), . . .</code>	指定した同じゲートを一度に追加可能
<code>qc.pauli('IXYZ',[0,1,2,3])</code>	各ビットごとに異なるパウリゲートを同時に追加可能
<code>qc.unitary(U,[0,1])</code>	任意のユニタリゲートを追加可能
<code>qc.append(sub_gate,[0,1])</code> <code>qc.append(HGate(),[1])</code>	- 既存の回路に別の回路をインプレースに直接結合（返り値はNone） <code>qc.to_instruction()</code> / <code>qc.to_gate()</code>で作成した再利用可能にした回路を追加する場合はこちらを使うこと
<code>qc.compose(sub_circ, qubits=[0,1,2,3])</code>	- 既存の回路に別の回路を結合し、デフォルトでは別の回路として返す <code>inplace</code>オプションの場合は直接結合し返り値はNone 結合先の量子ビットを指定する場合は<code>qubits</code>オプションを明示すること

回路を再利用可能（reusable）にする操作の違い

操作方法	非ユニタリ処理を含められるか	制御ゲート化 .control()の使用	用途
<code>qc.to_gate()</code>	不可	可能	まとめた回路を「一つのカスタム量子ゲート」としてさらに制御ゲート化や逆演算を行いたいとき
<code>qc.to_instruction()</code>	可能（測定、リセットなどを含められる）	不可	回路の一部をひとまとめにしてブラックボックス化・整理したいとき

TASK 1.2: Apply quantum operations 例題

30秒

13. You have built a two-qubit subcircuit `qc_b` and wish to create a controlled version of it with one control and two targets. Which sequence achieves this before appending to `qc_a`?

a.

```
gate = qc_b.to_gate().control()
qc_a.append(gate, [c, t0, t1])
```

b.

```
gate = qc_b.to_instruction().control()
qc_a.append(gate, [c, t0, t1])
```

c.

```
gate = qc_b.compose().control()
qc_a.append(gate, [c, t0, t1])
```

d.

```
gate = qc_b.control()
qc_a.append(gate, [c, t0, t1])
```

TASK 1.2: Apply quantum operations 正解

13. You have built a two-qubit subcircuit `qc_b` and wish to create a controlled version of it with one control and two targets. Which sequence achieves this before appending to `qc_a`?

a.

```
gate = qc_b.to_gate().control()
qc_a.append(gate, [c, t0, t1])
```

b.

```
gate = qc_b.to_instruction().control()
qc_a.append(gate, [c, t0, t1])
```

`to_instruction()` と `control()` は併用できない

c.

```
gate = qc_b.compose().control()
qc_a.append(gate, [c, t0, t1])
```

`compose()` の使い方が違う
`compose()` と `append()` は併用できない

d.

```
gate = qc_b.control()
qc_a.append(gate, [c, t0, t1])
```

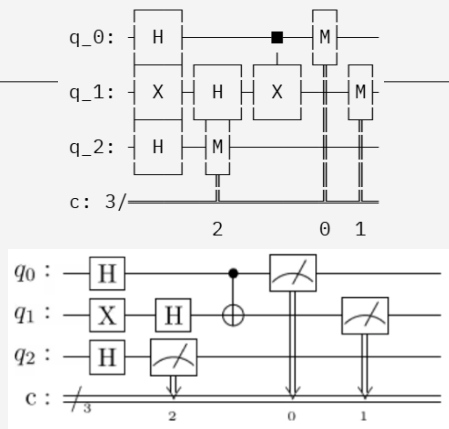
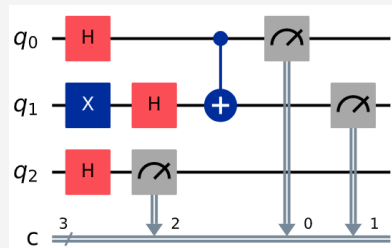
`to_gate()` の後に `control()` を入れるべき

- **Section 2: Visualize quantum circuits, measurements, and states**
 - TASK 2.1: Visualize quantum circuits

TASK 2.1: Visualize quantum circuits ポイント

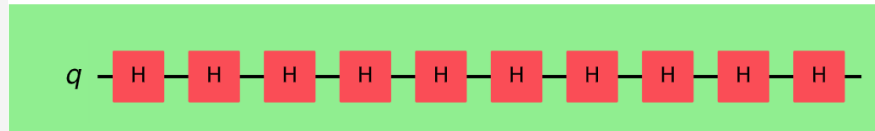
回路を描画する.draw()メソッドのオプションは以下の通り

- output
 - Text / latex / mplの3種類から選択可能
- filename
 - 指定したファイルパスに保存
- reverse_bits
 - 量子ビットの順番を上下にひっくり返すかどうか
- plot_barriers
 - バリアを表示するかどうか
- fold
 - 回路を指定した長さに表示を収める
- style
 - 表示の配色やフォントを指定できる
- scale
 - 描画サイズを調整できる



```
1 # Change the background color in mpl
2
3 style = {"backgroundcolor": "lightgreen"}
4 circuit.draw(output="mpl", style=style)
```

Output:



1. Which parameter of the `draw()` method specifies the output format of the quantum circuit visualization?

- a. `style`
- b. `output`
- c. `format`
- d. `backend`

TASK 2.1: Visualize quantum circuits 正解

1. Which parameter of the `draw()` method specifies the output format of the quantum circuit visualization?

a. `style`

出力の見栄えを調整するものであり、形式の指定ではない

☒ b. `output`

c. `format`

オプションに存在しない

d. `backend`

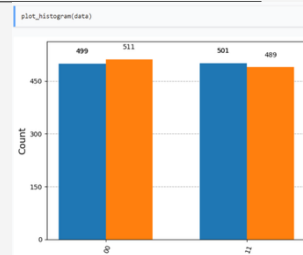
オプションに存在しない

- **Section 2: Visualize quantum circuits, measurements, and states**
 - TASK 2.2: Visualize quantum measurements

TASK 2.2: Visualize quantum measurements ポイント

■ plot_histogramの主なオプションは以下

- data: 測定結果を指定。辞書形式で {'001': 130} の形で渡す必要がある
 - stateオブジェクトから、sample_countsの出力でも可
- sort: ソート方針を指定
 - asc (ビット列の昇順) / desc (ビット列の降順) / value_desc (値の降順)
- legend: 凡例を指定
- title: グラフのタイトルを指定



■ 制御構文 (qc.if_test) 書き方

① 条件となる量子ビットの測定を行う

② withステートメントにqc.if_test((古典ビット, 値)) as else_:と書く

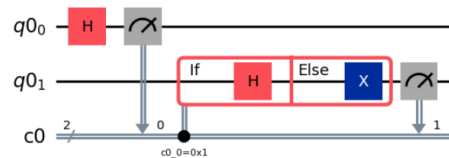
- Pythonコンテキストマネージャの文法に準拠
- 対応する古典ビットがその値の場合に発動
- elseを入れない場合はas else_は不要

③ with else_:でelse処理を追加する (任意)

```
from qiskit.circuit import QuantumCircuit, QuantumRegister, ClassicalRegister
qubits = QuantumRegister(2)
clbits = ClassicalRegister(2)
circuit = QuantumCircuit(qubits, clbits)
(q0, q1) = qubits
(c0, c1) = clbits

circuit.h(q0)
circuit.measure(q0, c0)
with circuit.if_test((c0, 1)) as else_:
    circuit.h(q1)
with else_:
    circuit.x(q1)
circuit.measure(q1, c1)

circuit.draw("mpl")
```



TASK 2.2: Visualize quantum measurements 例題

30秒

4. Which code fragment correctly measures `q0` into `c0` **mid-circuit** and applies `x(q1)` **only when** the measurement outcome is 1, before the final measurement of `q1`?

a.

```
qc.measure(0, 0)
qc.if_test((qc.clbits[0], 1)):
    qc.x(1)
qc.measure(1, 1)
```

b.

```
qc.measure(0, 0)
with qc.if_test((qc.clbits[0], 1)):
    qc.x(1)
qc.measure(1, 1)
```

c.

```
with qc.if_test((qc.clbits[0], 1)):
    qc.measure(0, 0)
    qc.x(1)
qc.measure(1, 1)
```

d.

```
with qc.if_else((qc.clbits[0], 1)) as else_:
    qc.x(1)
with else_:
    pass
qc.measure([0,1], [0,1])
```

TASK 2.2: Visualize quantum measurements 例題

4. Which code fragment correctly measures `q0` into `c0` mid-circuit and applies `x(q1)` only when the measurement outcome is 1, before the final measurement of `q1`?

a.

```
qc.measure(0, 0)
qc.if_test((qc.clbits[0], 1)):
    qc.x(1)
qc.measure(1, 1)
```

withステートメントが無い

b.

```
qc.measure(0, 0)
with qc.if_test((qc.clbits[0], 1)):
    qc.x(1)
qc.measure(1, 1)
```

c.

```
with qc.if_test((qc.clbits[0], 1)):
    qc.measure(0, 0)
    qc.x(1)
qc.measure(1, 1)
```

withステートメントの前に測定が無い

d.

```
with qc.if_else((qc.clbits[0], 1)) as else_:
    qc.x(1)
with else_:
    pass
qc.measure([0,1], [0,1])
```

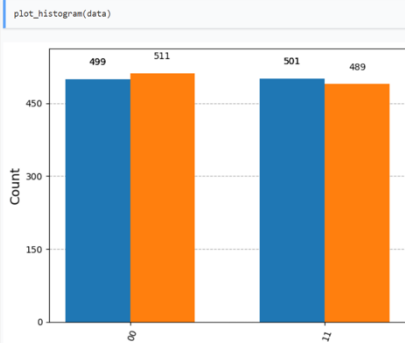
Withステートメントの前に測定が無い

- **Section 2: Visualize quantum circuits, measurements, and states**
 - TASK 2.3: Visualize quantum states

TASK 2.3: Visualize quantum states ポイント

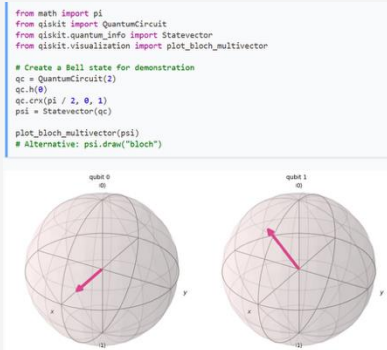
測定結果、量子状態の可視化は主に以下の通り

ヒストグラム



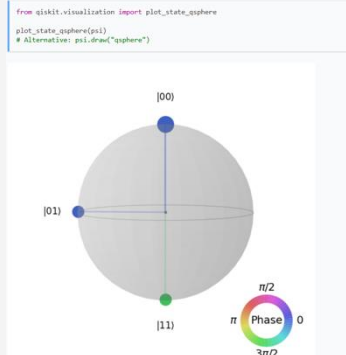
- 入力：辞書形式の測定結果
- 使い処：アルゴリズムの出力結果を確認

Bloch球



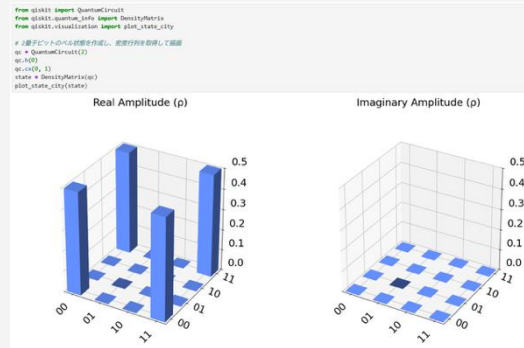
- 入力：状態ベクトル/密度行列
- 使い処：単一量子ビットへの量子操作を確認する
- 注意点：量子もつれ状態になると描画ができない

Qスフィア



- 入力：状態ベクトル/密度行列
- 使い処：量子もつれの状態を確認する
- 特徴：測定可能状態の確率振幅と位相を同時に表現

都市景観プロット



- 入力：状態ベクトル/密度行列
- 使い処：純粋状態および混合状態の密度行列の状態を表現

8. Given the following code:

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_state_qsphere

qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
state = Statevector.from_instruction(qc)
plot_state_qsphere(state)
```

Which of the following best describes the qsphere output?

- a. Two points at $|00\rangle$ and $|11\rangle$ with equal size, both colored the same.
- b. Two points at $|01\rangle$ and $|10\rangle$ with equal size, opposite colors.
- c. Four points at $|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$ equally distributed.
- d. A single point at $|00\rangle$ only.

TASK 2.3: Visualize quantum states 正解①

8. Given the following code:

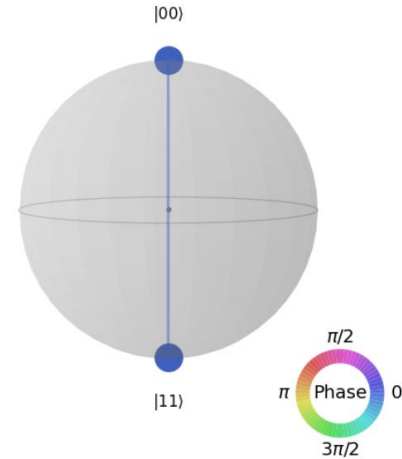
```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_state_qsphere

qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
state = Statevector.from_instruction(qc)
plot_state_qsphere(state)
```

$\frac{|00\rangle + |11\rangle}{\sqrt{2}}$

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_state_qsphere

qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
state = Statevector.from_instruction(qc)
plot_state_qsphere(state)
```



Which of the following best describes the qsphere output?

- ☒ a. Two points at $|00\rangle$ and $|11\rangle$ with equal size, both colored the same.
- ☐ b. Two points at $|01\rangle$ and $|10\rangle$ with equal size, opposite colors.
- ☐ c. Four points at $|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$ equally distributed.
- ☐ d. A single point at $|00\rangle$ only.

9. Given the following code:

```
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

state = Statevector([1/2, 1/2, 1/2, 1/2])
plot_bloch_multivector(state)
```

Which of the following describes the Bloch sphere plot?

- a. Both qubits point along +X axis.
- b. Both qubits point along +Z axis.
- c. First qubit along +X, second along -X.
- d. Random orientation with no symmetry.

TASK 2.3: Visualize quantum states 正解②

9. Given the following code:

```
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

state = Statevector([1/2, 1/2, 1/2, 1/2])
plot_bloch_multivector(state)
```

$$\frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2}$$

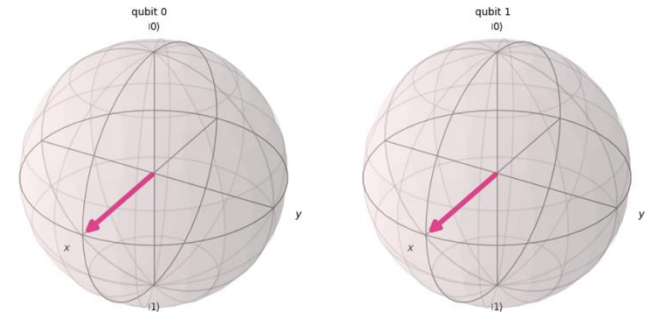
$$= \frac{(|0\rangle + |1\rangle)}{\sqrt{2}} \otimes \frac{(|0\rangle + |1\rangle)}{\sqrt{2}} = |+\rangle \otimes |+\rangle$$

Which of the following describes the Bloch sphere plot?

- ☒ a. Both qubits point along +X axis.
- ☐ b. Both qubits point along +Z axis.
- ☐ c. First qubit along +X, second along -X.
- ☐ d. Random orientation with no symmetry.

```
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

state = Statevector([1/2, 1/2, 1/2, 1/2])
plot_bloch_multivector(state)
```



END